Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – VI

Roll No.:T001	Name:Soham Acharekar
Class:TYIT	Batch:1
Date of Assignment:30/7/26	Date/Time of Submission:30/7/26

Part A - Theory:

What is a PWA?

A **Progressive Web Application (PWA)** is a web application that behaves like a mobile app. It can be installed on a device, loads quickly, and can even work offline using a Service Worker.

Remember Line:

"Website + App Features = PWA"

Components of a PWA

1. Web App Manifest (`manifest.json`)

The Manifest file stores information about the application such as its name, icons, theme color, and display mode.

Snippet:

```
{
  "name": "My Website",
  "short_name": "MyApp",
  "display": "standalone"
}
```

Remember Line:

"Manifest gives the App its Identity."

Mnemonic:

Name → Short → Start → Display → Background → Theme → Icons

2. Service Worker (`service-worker.js`)

A Service Worker runs in the background. It helps the application work offline, loads pages faster, and supports push notifications.

Snippet:

```
self.addEventListener("install", event => {
    console.log("Installed");
});
```

Remember Line:

"Service Worker Works in the Background."

3. Registering the Service Worker

The browser must register the Service Worker before it can be used.

Snippet:

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js');
}
```

Remember Line:

"Check → Register → Ready."

4. HTTPS

A PWA should run over HTTPS because it provides a secure connection between the browser and the server.

Remember Line:

"No HTTPS, No PWA."

Advantages of PWA

- Installable on desktop and mobile.
- Fast loading.
- Offline support.
- Responsive on all devices.
- App-like experience.

Remember Line:

"Fast, Secure, Installable, Offline."

Part B - Program

a. **Create a simple website for a College Information Portal and convert it into a Progressive Web Application.**

Code:

_Layout.cshtml | service-worker.js | **manifest.json** | HTMLtoPWA: Overview

Schema: https://www.schemastore.org/foxx-manifest.json

```
{
  "name": "HTML TO PWA",
  "short_name": "HTP",
  "start_url": "/",
  "display": "standalone",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "128x128",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    }
  ]
}
```

_Layout.cshtml | **service-worker.js** | manifest.json | HTMLtoPWA: Overview

HTMLtoPWA JavaScript Content Files | <global>

```
self Pin Tab ntListener("install", e => {
    console.log("Service worker installed");
});

self.addEventListener("fetch", e => {
    e.responseWith(fetch(e.request));
});
```

_Layout.cshtml | service-worker.js | manifest.json | HTMLtoPWA: Overview

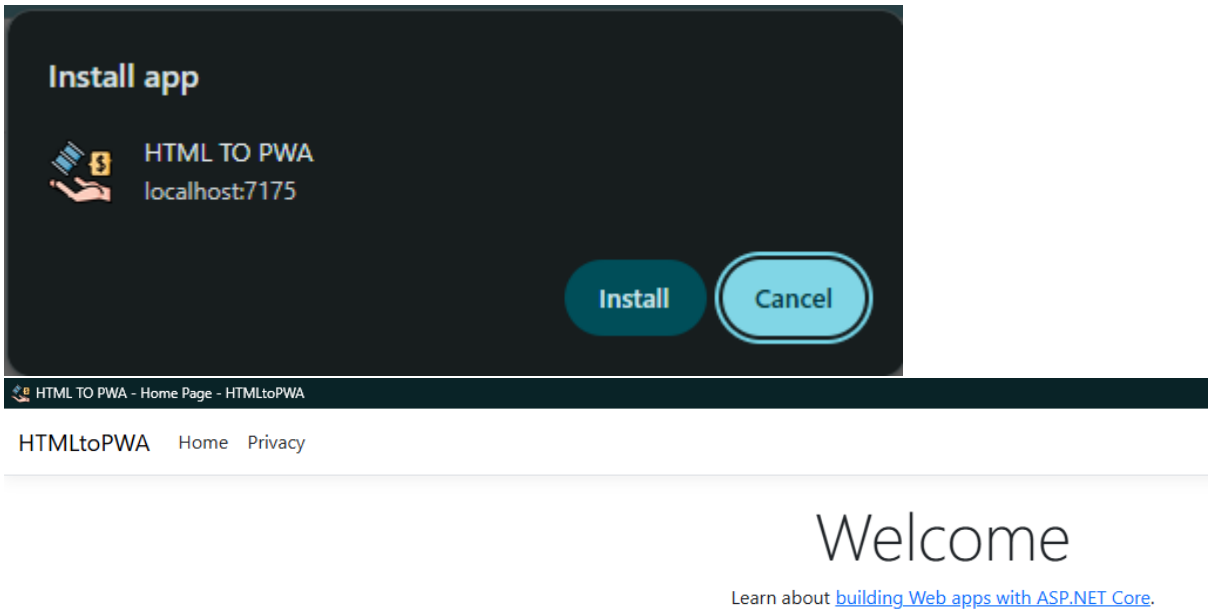
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - HTMLtoPWA</title>
    <script type="importmap"></script>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
    <link rel="stylesheet" href="~/HTMLtoPWA.styles.css" asp-append-version="true" />
    <link rel="manifest" href="/manifest.json" />
</head>
<body>
    <header>
        <
            The header element represents introductory content for its nearest ancestor sectioning content or sectioning root element. A header typically contains a group of introductory or
            <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">HTMLtoPWA</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-controls=
                    aria-expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                <ul class="navbar-nav flex-grow-1">
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
                    </li>
                </ul>
            </div>
        </div>
    </nav>
</header>
<div class="container">
    <main role="main" class="pb-3">
        @RenderBody()
    </main>
</div>
```

_Layout.cshtml | service-worker.js | manifest.json | HTMLtoPWA: Overview

```
            <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                <ul class="navbar-nav flex-grow-1">
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
                    </li>
                </ul>
            </div>
        </div>
    </nav>
</header>
<div class="container">
    <main role="main" class="pb-3">
        @RenderBody()
    </main>
</div>

<footer class="border-top footer text-muted">
    <div class="container">
        &copy; 2026 - HTMLtoPWA - <a asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
    </div>
</footer>
<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>
@await RenderSectionAsync("Scripts", required: false)
<script>
    if('serviceWorker' in navigator){
        navigator.serviceWorker.register('/service-worker.js');
    }
</script>
</body>
</html>
```

Output:



b. Create a PWA that displays a custom offline page when the user is not connected to the internet.

Code:offline.html :

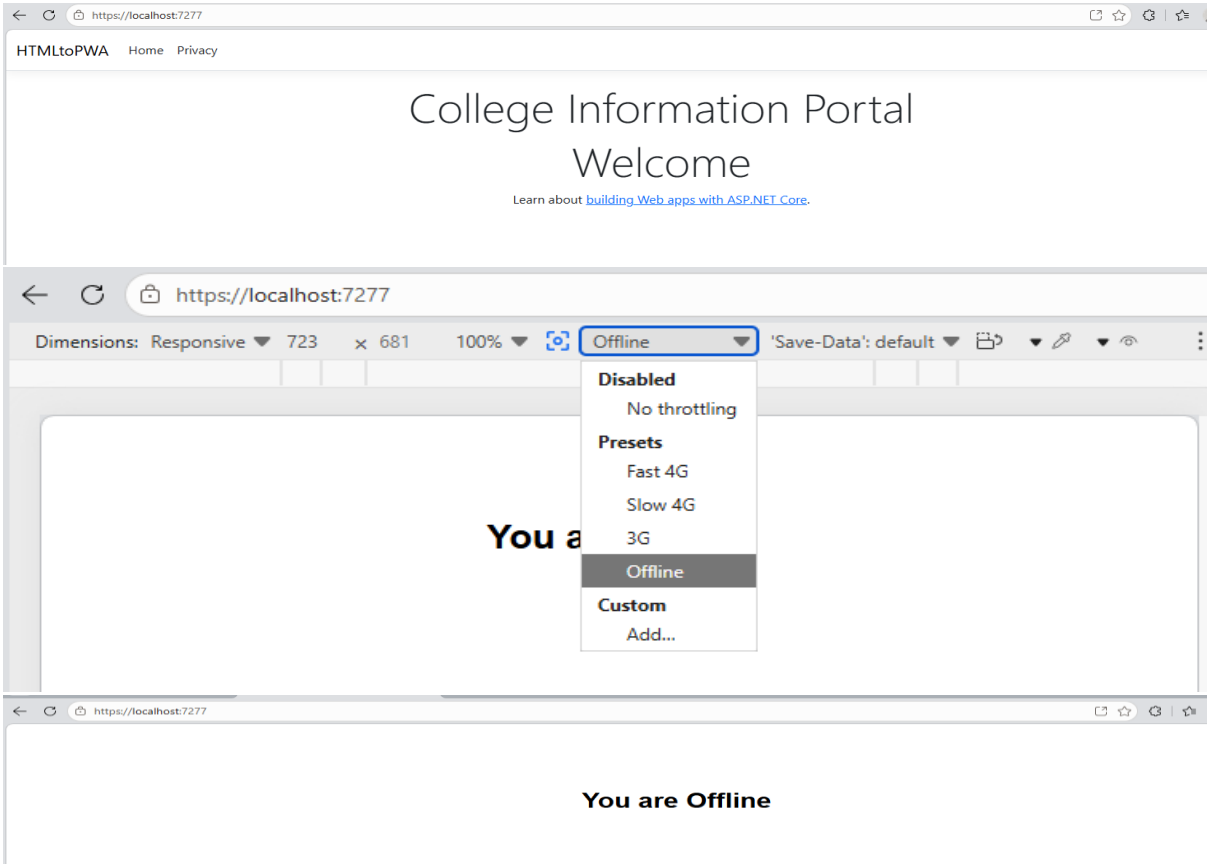
```
<!DOCTYPE html>
<html>
<head>
  <title>Offline</title>
</head>
<body style="text-align:center; margin-top:100px; font-family:Arial;">
  <h1>You are Offline</h1>
</body>
</html>
```

service-worker.js :

```
const CACHE = "v1";
self.addEventListener("install", event => {
  event.waitUntil(
    caches.open("offline-cache").then(cache => {
      return cache.addAll([
        "/",
        "/offline.html"
      ]);
    }));

  console.log("Service workers Installed ")
});
self.addEventListener("fetch", event => {
  event.respondWith(
    fetch(event.request).catch(() => {
      return caches.match("/offline.html");
    })
  );
});
```

Output:


https://localhost:7277
HTMLtoPWA
Home
Privacy
College Information Portal
Welcome
Learn about building Web apps with ASP.NET Core.
https://localhost:7277
Dimensions: Responsive
723
681
100%
Offline
'Save-Data': default
Disabled
No throttling
Presets
Fast 4G
Slow 4G
3G
Offline
Custom
Add...
You a
https://localhost:7277
You are Offline

Part C – Curiosity Question

Your college has a website that students use every day to check notices, timetables, and study materials.

Many students say:

"It would be easier if this website worked like a mobile app."

The college does not want to spend money on developing a separate Android or iPhone app.

Task

How can you convert the college website into an app-like experience that students can install on their phones?

Hint: Use a **Progressive Web Application (PWA)**.